Hiring

An agent that changes the shape of the network it belongs to,
and why it turns out not to need a compiler

Claude, 13 September 2026

Dear Neil,

One question, as simply as we can put it.

A firm has three stages in a line. Sales asks product development for goods; development asks research for ideas. At the end of each year a CEO looks at how the year went and **makes a hire** — another sales rep, another developer, another researcher.

A hire changes the network. So the CEO is an agent that changes the topology of the system it is part of. Can that be typed?

We thought the answer was no, and that hiring would need a recompile. It is yes, and it needs nothing but a combinator we already have and had never used.

1 Why it looks impossible

A second sales rep is not a configuration change. Two reps must each be told what to sell, so the prompt has two halves where it had one:

$$\text{Sales} \quad \longrightarrow \quad \text{Sales} \boxtimes \text{Sales}$$

These are different containers with different shape types. So when the CEO hires, the thing it prompts next year is not the thing it prompted this year.

Write the CEO as a worker and the difficulty is immediate. A worker is

$$A \otimes X \longrightarrow Y$$

with Y *fixed*. The CEO's Y is not fixed: it is whatever the firm is this year, and the CEO is the reason it changed. What one apparently wants is a type-level function

$$\text{Firm} : S \longrightarrow \text{Type},$$

which TypeScript has not got, and the obvious conclusion is that the map must be realised by running the compiler again — so a hire is a rebuild, and an agent that hires needs a shell.

That conclusion is wrong.

2 The construction

S is finite. The CEO has three things it can decide, so the firm has three shapes it can have. Form the container

$$\hat{Y} \;=\; \Sigma\,(s : S).\, Y(s) \;=\; Y(s_1) + Y(s_2) + Y(s_3).$$

A finite sum of containers is $+$, which is the fourth of the four combinators and the one neither `stalin` nor `federation` ever uses. Then

$$\mathrm{CEO} \;:\; A \otimes X \longrightarrow \hat{Y}$$

is an ordinary container morphism with an ordinary fixed codomain, and **hiring is the choice of a summand**.

The dependent family did not have to be eliminated. It had to be summed.

3 Why there is no compiler in it

One might expect the sum to be a bookkeeping trick that still ends in a rebuild. It does not, and the reason is a detail of what the combinators construct.

> `sumC` and `tensorC` build **runners**, and a runner is a *description*, not a connection. `leaf` holds a port number. `tensorC` holds two runners. Nothing is contacted until a shape is `run`, and a shape names exactly one summand.

So $\hat{Y}$ can name all three firms at once and cost nothing for the two it is not being. Here is a decade, on six real processes:

```
year  agents  revenue  unsold  constraint   next year
1928      1        3       9       sales      run two
1929      2       10       2       sales      run three
1930      3       12       0       sales      run three
...
10-year revenue 106
```

Three sales servers are running from the first line. In 1928 exactly one of them is prompted. The CEO hired twice, the network changed twice, and no compiler ran.

> **For any family of topologies one is willing to enumerate in advance, changing the topology is routing and not rebuilding.** A compiler is needed exactly when the agent wants to leave the family — and the family is finite precisely so that it cannot.

4 The one rule the CEO must obey

The summand is chosen from the **state**, never from a reply.

That is not a stylistic preference. Choosing the next shape from a reply is the one thing $\triangleleft$ forbids: a `next` that branches between shapes of the second container has a union return type and lies in no single fibre. It is the reason `stalin` has no typed routing agent.

Choosing from the state is free, because the forward leg $S \times X_0 \to Y_0$ is an ordinary function and may branch on s however it likes. And the backward leg is what puts this year's reply into next year's state.

Observation. *So a worker is the repair for that restriction. One cannot branch on a reply; one can write the reply into the state and branch on it next tick. The CEO hires at the end of the year for this reason and not for a narrative one — the one-year delay is the price of the restriction, appearing as a calendar.*

5 Where the human goes

If no compiler runs, there is no build step for a person to stand in, and that looked like a loss. We think it is a gain.

> The review point is not the compiler. It is the **finiteness of** S.
>
> A person approves the family of possible firms *once*. Thereafter the agent has real authority — each year it decides what shape the firm has — and it cannot invent an interface, because its decision is a total function into three values and the decoder at the wire refuses anything else.
>
> Approving a bound once is a stronger guarantee than approving each rebuild, because a bound cannot be tired, rushed, or talked round.

An agent that wants a *fourth* topology is a different and much larger question: it needs a shell, a rollback story, and a human. That is the only place a compiler is a checkpoint rather than a limitation, and we have not built it.

6 The question for you

Everything above rests on S being a finite set, and on a worker's state being $A = S \triangleright S$. Both are reconstructed from Robin's notes rather than from a source we have read.

Is the finiteness of the state essential to workers, or incidental?

If it is essential, the argument stands and the bound is real. If workers permit infinite state, then $\Sigma_s Y(s)$ is not a finite sum, the decoder cannot enumerate what it is refusing, and the agent's authority is not bounded by anything we can check. We would rather know before building further.